

Title	openrisc_ahb
Project	
Description	AHB를 사용하도록 변경된 OpenRISC platform에 대한 설명
Author	holelee
Date	8/Sep/2005

(*) AHB를 사용하도록 변경

OpenRISC or1200은 기본적으로 WISHBONE bus를 사용하도록 되어 있는데 BUS interface를 AHB로 변경하여 다른 곳에 사용하기 쉽도록 하였다.

(*) AHB로 변경한 방법

WISHBONE은 size 정보가 없는 대신 byte selection 신호를 사용하고, burst transfer에 관한 signal이 부족하다. 또한 address/data가 pipeline되어 있지 않은 구조를 가지고 있어서 일반적인 WISHBONE to AHB bridge를 설계하는 것이 쉽지 않다. 따라서 bridge를 설계하는 방법 대신에 or1200 cpu core에서 나오는 RISC bus interface를 AHB로 변환하는 방법을 사용하여 설계하였다. 사실 RISC bus interface도 WISHBONE과 크게 다르지는 않는데, 다음과 같은 특징을 사용하여 설계를 간단하게 할 수 있었다.

- cab 신호가 high이면 항상 wrap burst 4 WORD transfer(cache line size)이어서 다음 address를 받지 않고도 계산해 낼 수 있다.
- instruction interface의 size는 항상 WORD이다.
- Write transfer는 burst transfer가 아니다. : or1200은 write-through cache를 사용하므로 cache line 전체를 bus로 write하는 burst transfer는 존재하지 않는다.

다만, 일반적으로 사용하는 little endian AHB interface가 아니라 big endian AHB interface를 사용하도록 하였다. 이는 or1200이 기본적으로 big endian machine이기 때문이다.(참고로 OpenRISC architecture는 big/little endian 모두를 support할 수 있도록 되어 있다.)

(*) 변경된 RTL code

or1200의 RTL code 중에 다음과 같은 file이 변경되었다.

- or1200_wb_biu.v => or1200_ahb_biu.v : data bus interface unit
- or1200_iwb_biu.v => or1200_iahb_biu.v : instruction bus interface unit
- or1200_lsu.v => or1200_lsu_ahb.v : load store unit
- or1200_cpu.v => or1200_cpu_ahb.v : cpu core top module
- or1200_dc_top.v => or1200_dc_top_ahb.v : data cache top
- or1200_sb.v => or1200_sb_ahb.v : store buffer
- or1200_top.v => or1200_top_ahb.v : top module

or1200_ahb_biu.v와 or1200_iahb_biu.v 파일이 AHB interface를 구현한 부분이고, 나머지 파

일들은 or1200_lsu.v에서 byte selection 신호뿐 아니라 size 정보를 출력하여 or1200_ahb_biu.v로 연결하기 위한 변경되었다. or1200_top_ahb.v는 or1200의 top module로 기존의 WISHBONE bus를 위한 port가 모두 AHB bus를 위한 port로 변경되었다. 나머지는 변경하지 않고 사용하였으므로 MMU/Cache/store buffer 등을 On/Off 시키기 위해서는 기존의 or1200과 마찬가지로 or1200_defines.v를 변경하면 된다.

(*) Test Platform

변경된 or1200을 test하기 위해 AHB platform을 만들었다. AHB platform은 Example AMBA System(EASY)를 기반으로 약간 변경하여 4 master/4 slave AHB bus를 구성하였다. 4 개의 master port중 2개의 master port에 or1200의 instruction/data interface를 연결하였으며 나머지 port는 막아 두었다. 4개의 slave 중 두 곳에 Synchronous SRAM controller를 연결하였다. 나머지 두 slave port는 default slave에 연결하였다.

Address map은 다음과 같다.

- SRAM0(0x00000000~0x00FFFFFF : 512Kx32bit RAM을 사용) : program/data 저장
- SRAM1(0x04000000~0x04FFFFFF : 512Kx32bit RAM을 사용) : not used
- 나머지 region : HSEL2에 연결된 default slave

Synchronous SRAM controller는 거의 항상 hready가 high인 slave로 or1200의 AHB interface를 테스트하는데 무리가 있다고 생각하여 hready가 random하게 생성되도록 변경(ismc_ahb.v => ismc_ahb_random.v)하여 test를 진행하였다.

(*) test software

사용한 test software는 open risc platform(ORP)에서 사용하는 test program중에 UART만을 제외하고 basic/cbasic/dhry/mul/mmu/except/syscall/tick 을 사용하였다. 또한 AHB에서 전달되는 error response가 제대로 bus error exception을 발생시키는지 보기 위해 새로 작성된 i_buserr/d_buserr라고 하는 간단한 프로그램을 새로 작성하였다.

기본적으로 ORP와 AHB test platform사이에는 address map에 차이가 존재한다. ORP는 초기 reset 시에는 flash가 0x00xxxxxx 영역에 매핑되어 booting이 시작되고, 특정 register bit를 바꾸어서 flash를 0x04xxxxxx로 매핑을 바꾸고, 0x00xxxxxx에는 RAM이 새롭게 매핑되어 flash에 있는 program과 data를 RAM으로 copy하도록 되어 있다. AHB test platform도 동일하게 만들 수 있으나 상당히 번거롭기 때문에 그냥 0x00xxxxxx에 있는 SRAM에서 booting이 이루어지도록 하였다. 이러한 차이 때문에 link script(orp.ld)가 변경되었고, exception vector가 존재하는 assembly 파일들을 변경하여 exception vector의 address를 조정하였다.

참고로 Test software는 exit라는 함수를 호출함으로써 simulation의 종료를 알리게 된다. exit함수는 실제로 “l.nop 1”이라는 instruction을 수행할 뿐이고, simulation의 종료는 testbench 디렉토리 밑에 있는 or1200_monitor.v에서 or1200이 수행하는 instruction을

monitoring하고 있다가 “l.nop 1”이 있으면 \$finish로 simulation을 종료시키게 된다.

(*) test 수행 방법

다음과 같이 수행하면 간단히 이루어진다.

```
$ cd sim/run
```

```
$ ../bin/compile_sw.sh
```

```
$ ../bin/run_test.sh
```

compile_sw.sh은 test용 software를 build하는 script로, 동작하기 위해서는 OpenRISC용 cross tool chain(or32-uclinux)이 제대로 설치되어 PATH에 잡혀 있어야 한다. run_test.sh은 modelsim을 이용하여 simulation을 수행하므로 modelsim용 환경도 미리 잡혀 있어야 한다.

(*) 기타 중요한 사항

or1200의 RISC bus interface는 bus transfer를 response를 받지 않은 상태에서 중간에 그만 둘 수 있다. cyc, stb 신호가 high가 되어 transfer가 시작되었다가, ack 신호를 받지 않은 상황에서 cyc, stb가 다시 low로 바뀔 수도 있다. Data access가 중간에 abort되는 경우에는 register access에서 문제가 된다. Test 프로그램을 돌려본 결과를 보면 data access가 abort되는 경우에는 data cache가 enable되고 data access에서 error가 발생하였을 때 abort되는 경우가 생긴다. 그 외의 abort는 모두 instruction access이다. Data access에서 error가 발생했을 때 abort되는 것은 잘못된 region을 access하였을 때 뿐이므로 큰 문제가 되지 않는다.

or1200은 CPU clock이 WISHBONE bus clock에 비해 2배 혹은 4배 높은 경우를 지원하는데, AHB로 변경된 or1200은 그런 것을 지원하지 않는다. or1200의 clock이 AHB의 clock보다 빠르거나 느리게 동작하도록 하기 위해서는 외부에서 맞추어 주도록해야 한다.

(*) AHB의 제한 사항

설계된 AHB bus interface는 다음과 같은 제약 사항을 가진다.

- big endian을 사용한다. 일반적으로 AHB는 big endian과 little endian을 모두로 설계될 수 있지만, 일반적으로 little endian으로 설계된다. OpenRISC도 bi-endian architecture인데, or1200의 내부는 big endian을 사용하도록 되어 있다. 따라서 우선은 big endian을 사용하도록 하였다. Little endian AHB bus를 사용하는 경우에 대해서는 or1200을 변경하여 little endian으로 변경하던지 BUS level에서 endianness를 변경하는 부분을 설계해서 넣을 필요가 있다. or1200 hardware를 little endian으로 변경하는 것은 쉬우나 사용하는 gcc cross compiler도 모두 little endian으로 바꾸고 그것을 검증해야 하는 부담이 있다. BUS에서 endianness를 변경하는 부분을 설계하는 것도 hardware 설계는 비교적 쉽지만, BUS에 붙어 있는 memory에 or1200용 code/data를 loading할 때 문제가 발생할 수 있다. 따라서 주의를 필요로 한다.
- hgrant 신호를 받은 후 burst transfer가 시작된 후 transfer가 다 끝나지 않았을 때 hgrant

신호가 low로 가서 BUS의 소유권을 잃었을 때는 오동작을 하게 된다. 보통의 AHB arbiter는 일반적으로 undetermined length burst transfer를 제외하고는 burst transfer 중간에 master를 바꾸지 않는다. Or1200 AHB interface는 burst transfer로 항상 WRAP4만을 사용하기 때문에 큰 문제가 되지 않는다.

- hprot 신호를 encoding하지 않고 항상 0으로 내보낸다. BUS에 Level 2 Cache를 붙이지 않는 이상 필요하지 않으므로 큰 문제가 되지 않는다.
- Retry/split을 지원하지 않는다. Retry/split response가 왔을 때, 지금은 bus error가 되어 있다. ARM이 만든 아주 특이한 slave를 제외하고는 retry/split을 지원하는 AHB slave는 없을 것이므로 큰 문제가 되지는 않을 것이다.

(*) 사용 방법

- OpenRISC AHB top module integration guide

top module의 port는 다음과 같다.

```
module or1200_top(  
    // System  
    clk_i, rst_i, pic_ints_i,  
  
    // Instruction AHB INTERFACE  
    ihbusreq_o, ihgrant_i,  
    ihaddr_o, ihtrans_o, ihwrite_o, ihsize_o, ihburst_o, ihprot_o,  
    ihwdata_o, ihrdata_i, ihready_i, ihresp_i,  
  
    // Data AHB INTERFACE  
    dhbusreq_o, dhgrant_i,  
    dhaddr_o, dhtrans_o, dhwrite_o, dhsize_o, dhburst_o, dhprot_o,  
    dhwdata_o, dhrdata_i, dhready_i, dhresp_i,  
  
    // External Debug Interface  
    dbg_stall_i, dbg_ewt_i, dbg_lss_o, dbg_is_o, dbg_wp_o, dbg_bp_o,  
    dbg_stb_i, dbg_we_i, dbg_adr_i, dbg_dat_i, dbg_dat_o, dbg_ack_o,  
  
    `ifdef OR1200_BIST  
        // RAM BIST  
        mbist_si_i, mbist_so_o, mbist_ctrl_i,  
    `endif
```

```

`endif

// Power Management

pm_cpustall_i,
pm_clk_sd_o, pm_dc_gate_o, pm_ic_gate_o, pm_dmmu_gate_o,
pm_immu_gate_o, pm_tt_gate_o, pm_cpu_gate_o, pm_wakeup_o, pm_lvolt_o
);

```

우선 clk_i와 rst_i는 AHB clock과 Reset 신호를 연결해 주면 된다. 단 rst_i는 active high reset이므로 HRESETn을 직접 연결하면 안되고, inverting해서 연결해 주어야 한다. pic_ints_i는 interrupt controller에 연결하게 될 외부 interrupt 입력으로 default상태에서는 20 bit의 길이를 가지게 된다.

Instruction/data AHB interface는 각각 prefix로 i와 d가 붙어 있는 AHB master signal이므로 그것을 외부 BUS에 연결하면 된다. AHB2AXI interface에 연결할 때에는 hgrant_i를 항상 1로 연결하면 된다.

External Debug Interface 신호는 외부의 debug unit을 붙일 때 사용하는 신호로 보인다. ORP(OpenRISC soc Platform)에서는 외부에 JTAG interface와 WISHBONE bus interface를 가지는 debug unit이 붙어 있는데 크게 사용될 것으로 보이지 않아서 수정하지 않았다. 외부에 debug unit을 붙이고 싶지 않을 때는 모든 입력 신호를 0으로 묶어 두고, 출력 신호는 open하면 된다.

Power Management 관련 신호는 or1200_top에 들어 있는 power management unit에 연결되는 신호로 power management unit을 분석하지 않아서 어떻게 동작하는 알 수 없다. 사용하고 싶지 않다면, or1200_top내부에 있는 power management unit을 제거하거나, 제거가 귀찮으면 pm_cpustall_i 신호를 0으로 묶어 두고 나머지 출력 신호들은 open하면 된다.

- Software 작성 방법

Test software가 들어 있는 sw 디렉토리의 내용을 참조하여 만들면 된다. sw/support 디렉토리 밑에 있는 reset.S에 reset vector가 들어있고, sw/support 디렉토리에 있는 orp.ld가 link script이다. sw/cbasic 디렉토리에 있는 cbasic.c 등을 참조하면 큰 어려움 없이 사용자가 원하는 software를 작성할 수 있으리라 생각된다.